

M2 Data Science – M2 Intelligence Artificielle

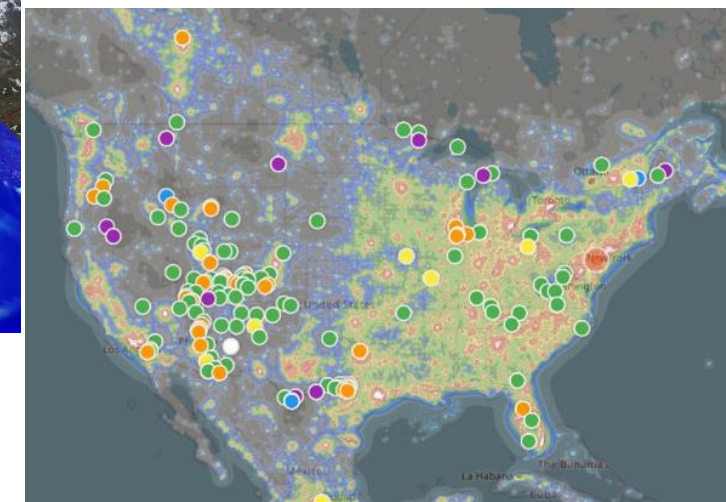
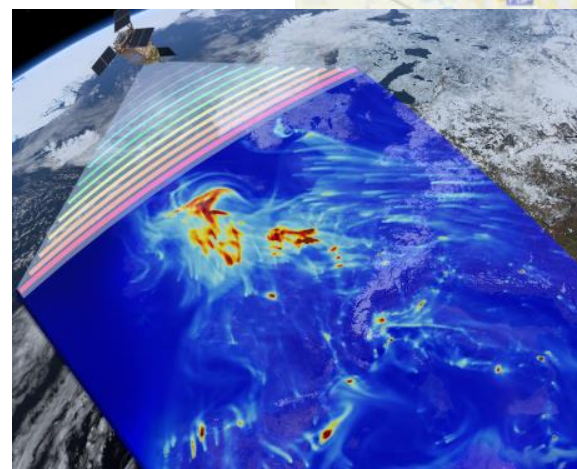
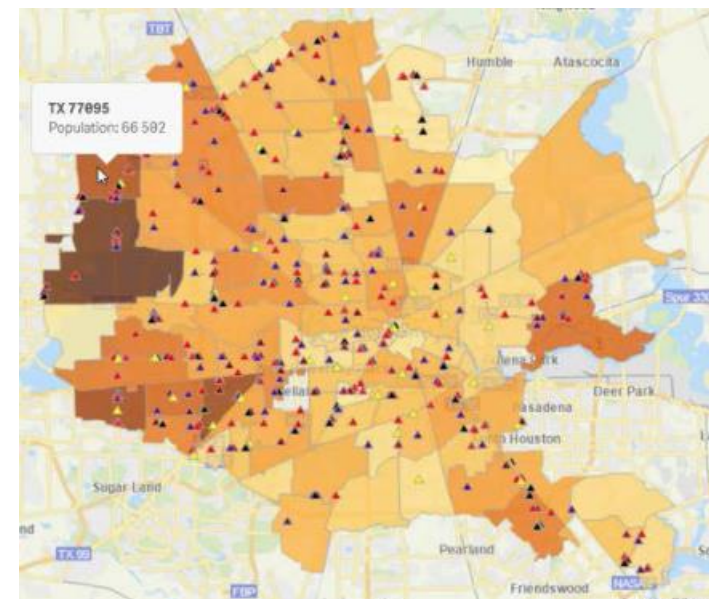
# Data Mining

Miner des données  
spatiales et temporelles



# Définitions: données spatiales

- Les **données spatiales** ou **géo-localisées** ont pour particularité d'être liées à un lieu précis
  - Soit directement via des coordonnées  $(x, y)$ 
    - Ex. longitude et latitude
  - Soit indirectement via un lieu plus ou moins précis
    - Ex. Adresse, quartier, ville, régions, pays, etc. (Granularité)
    - Peut nécessiter des transformation, lieu  $\rightarrow$  coordonnées
- Dans la vie réelle, plusieurs sources possibles:
  - Capteurs
  - Satellites
  - Études sociologiques (ex. votes)
  - Etc.
- Nécessite un traitement particulier pour repérer des dynamiques localisées



# Miner des données spatiales

Place à la pratique !

- Sur [kaggle](#), téléchargez l'un des jeux de données suivants:
  1. [GeoPlant: Spatial Plant Species Prediction Dataset](#)
  2. [US Wildfire Dataset \(2014-2025\)](#)
  3. [World Air Quality Index by City and Coordinates](#)
  4. [Biodiversity in National Parks](#)
  5. [Toronto Apartment Rental Price](#)
  6. [The Chernobyl Disaster Air Concentration](#)
  7. [Air quality Frankfurt Dataset](#)
- Vérifiez et nettoyez vos données !
- Discutez des informations que vous pouvez extraire de ces données
  - Comment les représenter ?
  - Comment les regrouper ?
- Faites des tests et discutez de vos résultats



# Visualisation

## Points « d'intérêts »

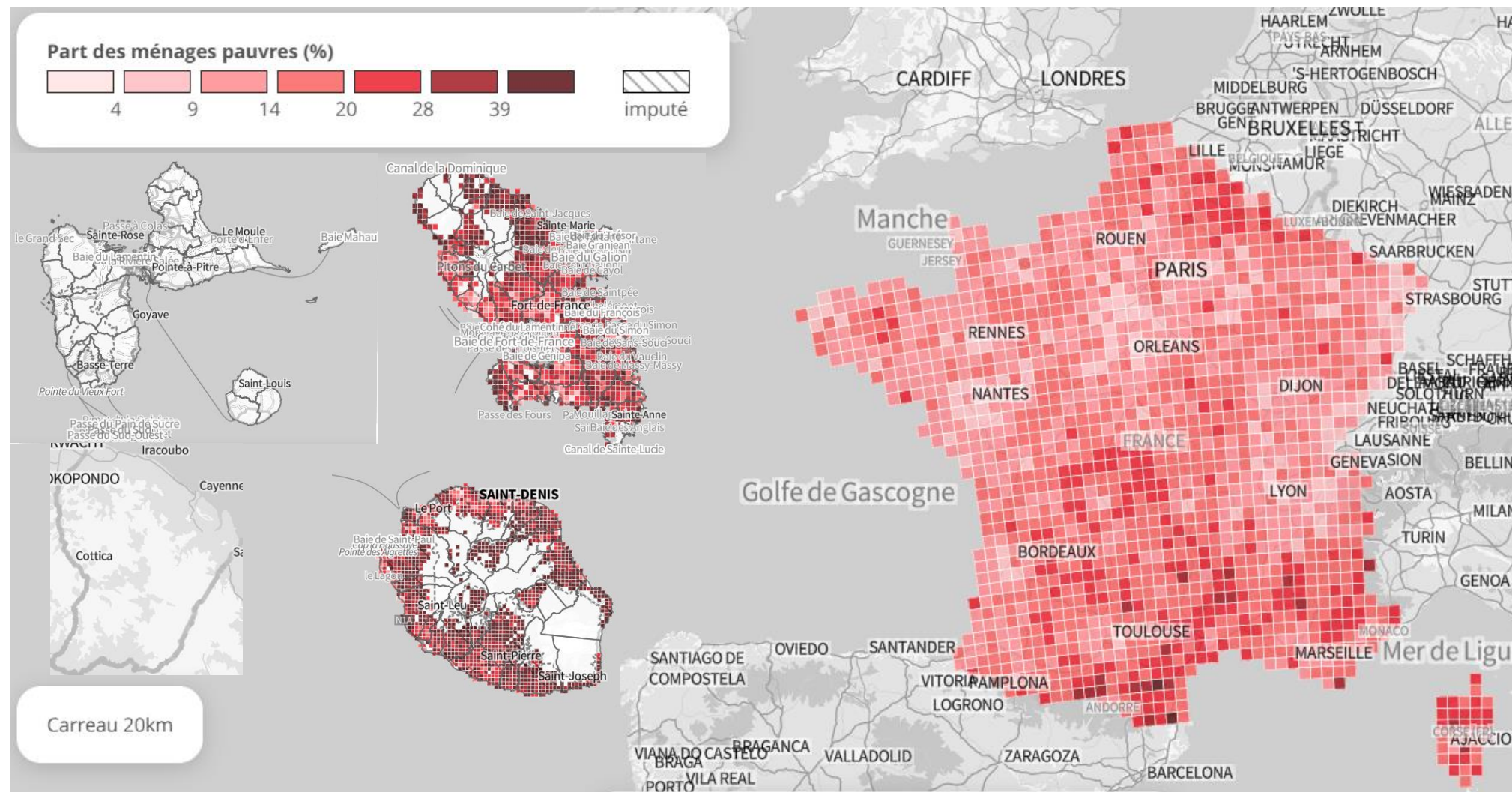


- Points à la position exacte
  - Taille du point basée sur une variable d'intérêt
- Peut vite être illisible avec beaucoup de points
- Ne rend pas nécessairement compte de particularités locales

Source: <https://feuxdeforet.fr/cartes/feux/>  
(le 24/07/2026)

# Regroupement

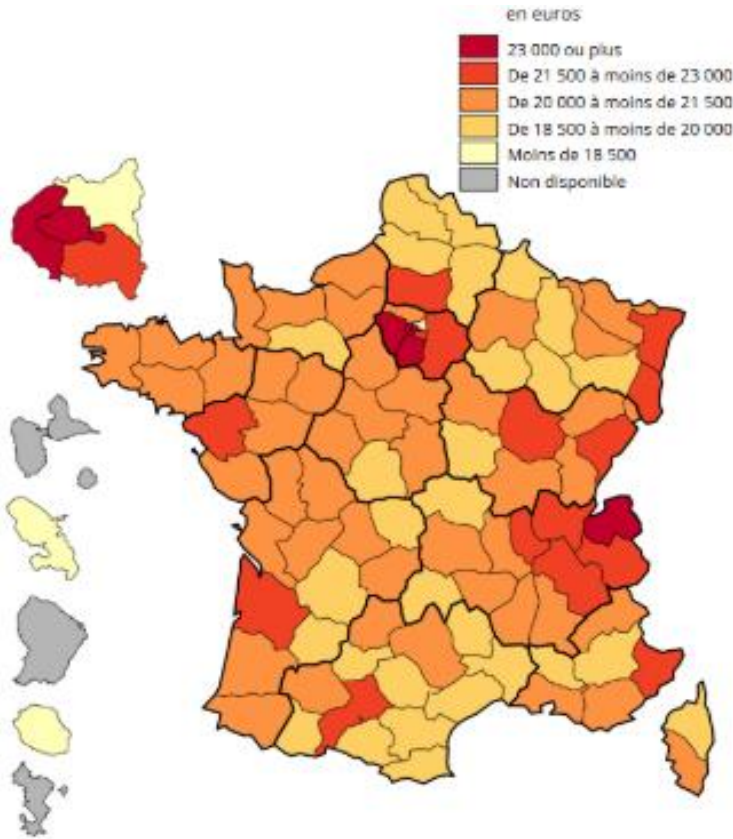
## Découpage en carreaux



Source: <https://www.insee.fr/fr/outil-interactif/7737357/map.html>

# Regroupement

## Régionalisation prédéfinie



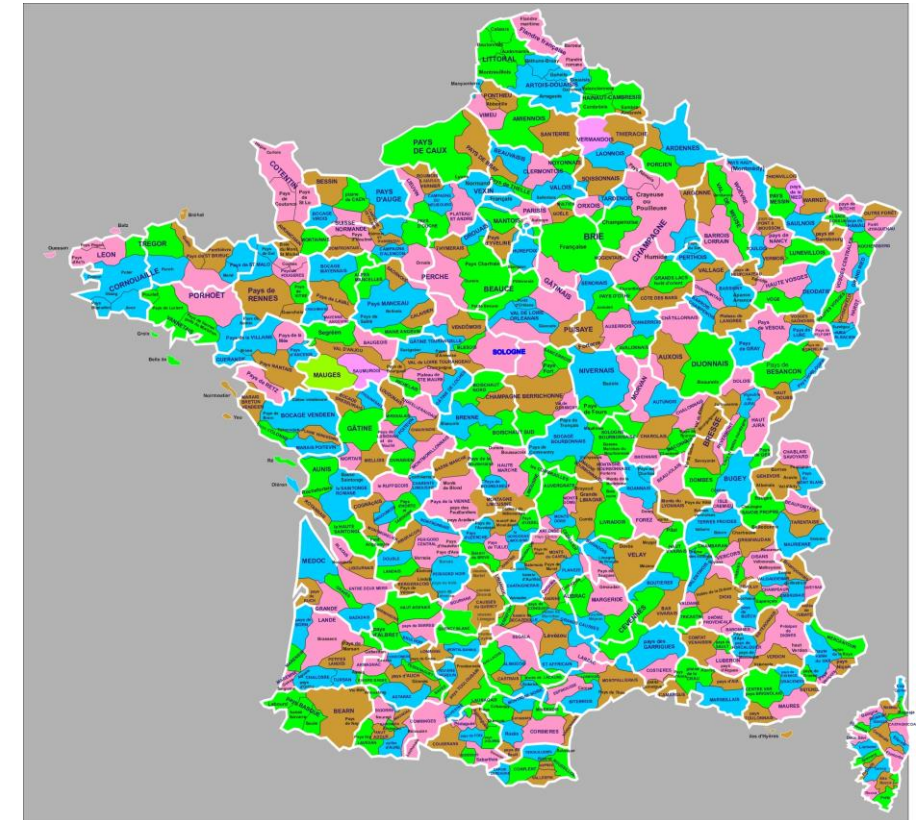
Découpage administratif

### Problèmes:

- Découpage arbitraire
  - Autres découpages possible -->
- Invisibilise des dynamiques aux zones limitrophes

### Donc:

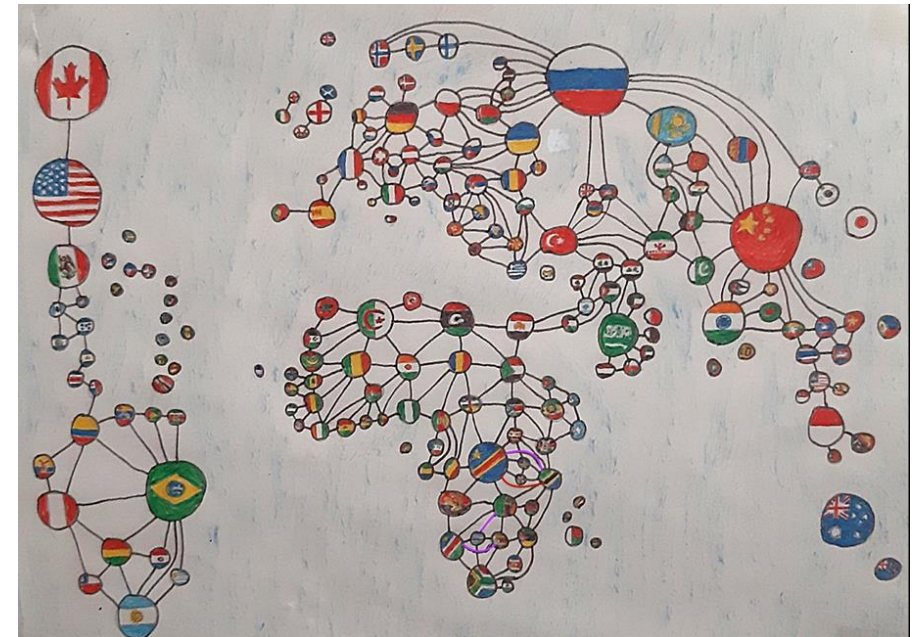
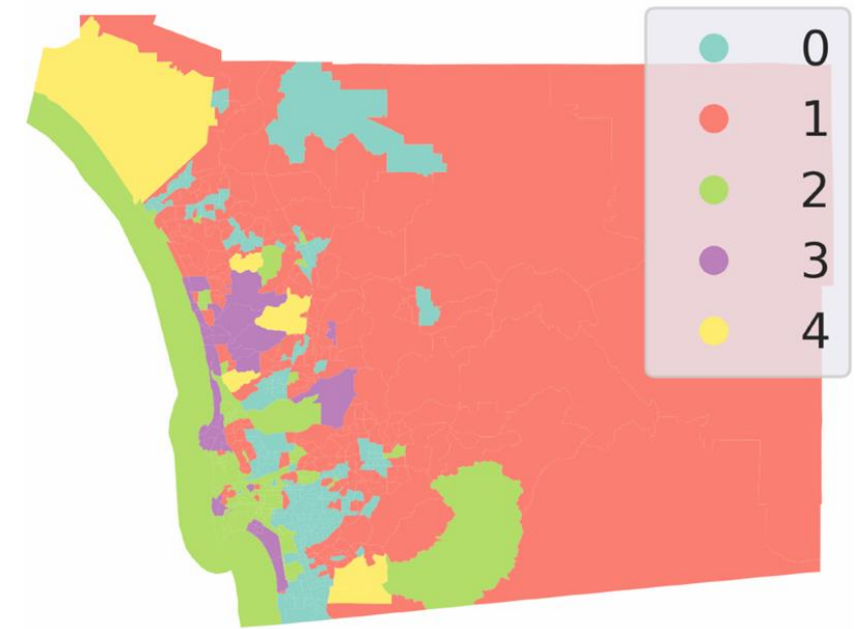
⇒ Il serait plus intéressant de créer directement des régions basées sur nos variables d'intérêt



Régions "naturelles" de France

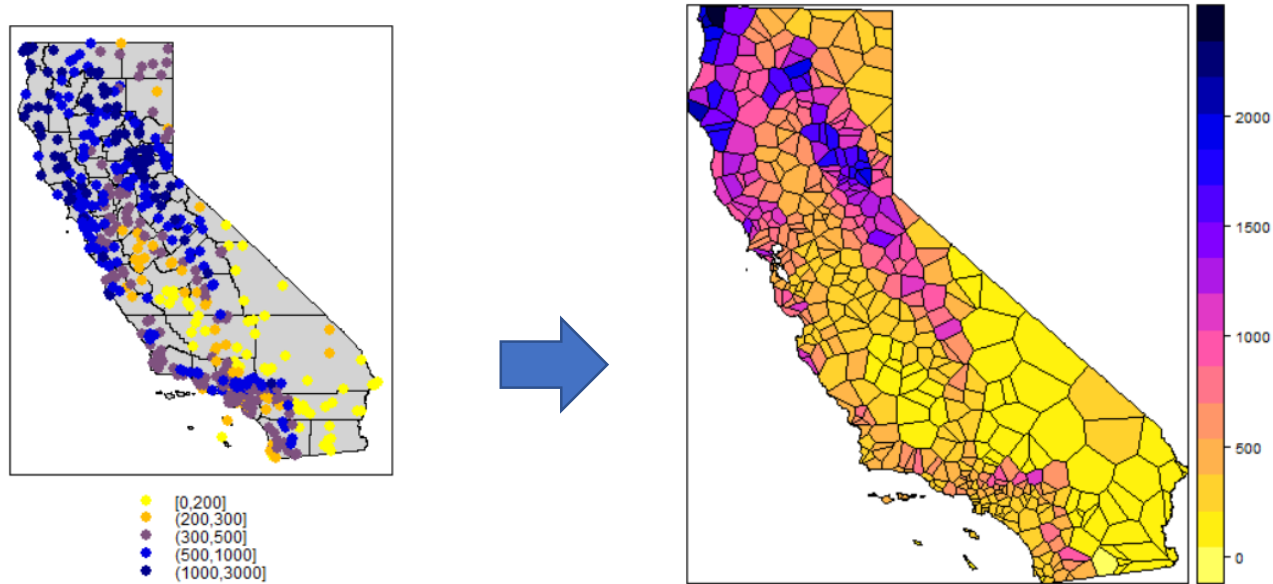
# Regroupement Régionalisation

- **Objectif:** regrouper des points partageant des caractéristiques similaires pour créer des zones, des « régions »
- Très similaire à du **clustering**
- Mais, pour trouver des *clusters* spatiaux, on ne veut regrouper que les données **spatialement contiguës**
- Solution possible:
  - Traiter les points comme un **graphe**, un maillage de points
  - Appliquer des algorithmes de regroupement de nœuds



# Regroupement

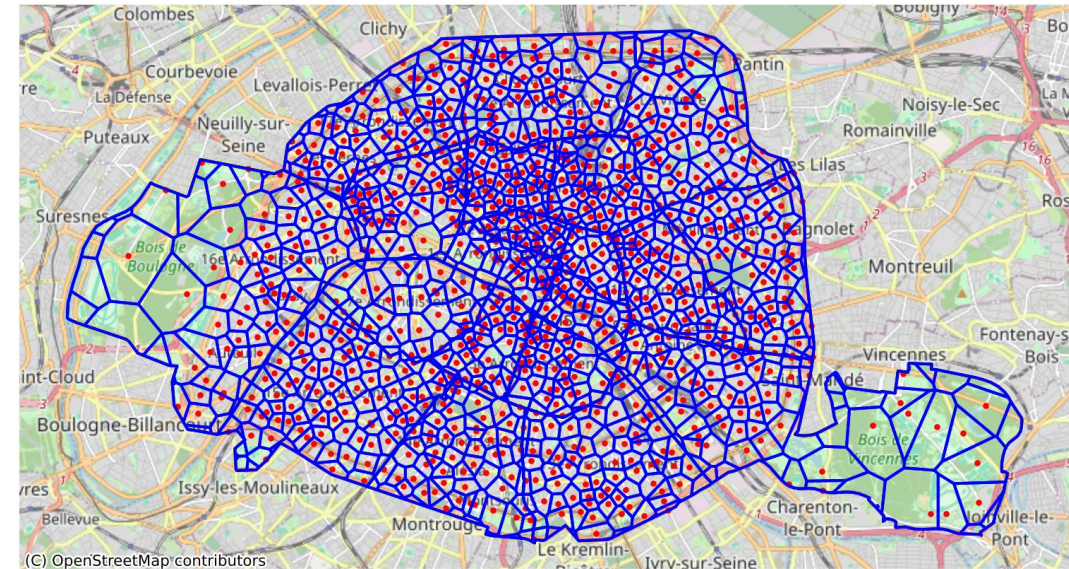
## Maillage ou Tessellation



**Objectif:** créer une zone autour de chaque point

=> Tessellation de Voronoï

**Exemple:** répartition des stations vélib à Paris



# Statistiques spatiales

## Auto-correlations

- **Problématique:**
  - Est-ce les valeurs observées sont aléatoires ou corrélées à leurs positions dans l'espace et leur voisinage
- **Objectif:**
  - Mesurer la similarité (corrélation) entre des observations «proches»
- Se base généralement sur l'**index I de Moran**



W=

|   |   |   |   |   |
|---|---|---|---|---|
| 0 | 1 | 0 | 1 | 1 |
| 1 | 0 | 1 | 1 | 1 |
| 1 | 1 | 0 | 0 | 1 |
| 1 | 1 | 0 | 0 | 0 |
| 1 | 1 | 1 | 0 | 0 |

Y=

|    |    |    |    |    |
|----|----|----|----|----|
| 17 | 12 | 15 | 14 | 13 |
|----|----|----|----|----|

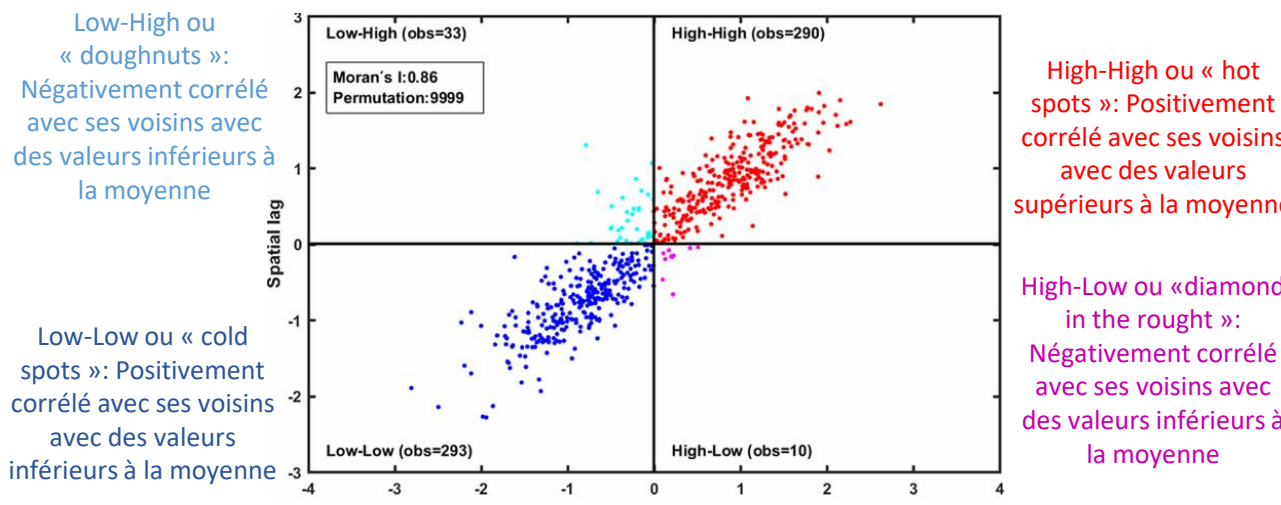
Nombre d'observations

Si il y a un lien entre  $i$  et  $j$   
(ou poids du lien)

$$I = \frac{n}{\sum_{i=1}^n (y_i - \bar{y})^2} \frac{\sum_{i=1}^n \sum_{j=1}^n w_{ij} (y_i - \bar{y})(y_j - \bar{y})}{\sum_{i=1}^n \sum_{j=1}^n w_{ij}}$$

Somme des distances entre les valeurs observées et la moyenne des valeurs

• **Interprétation: Moran's plot**



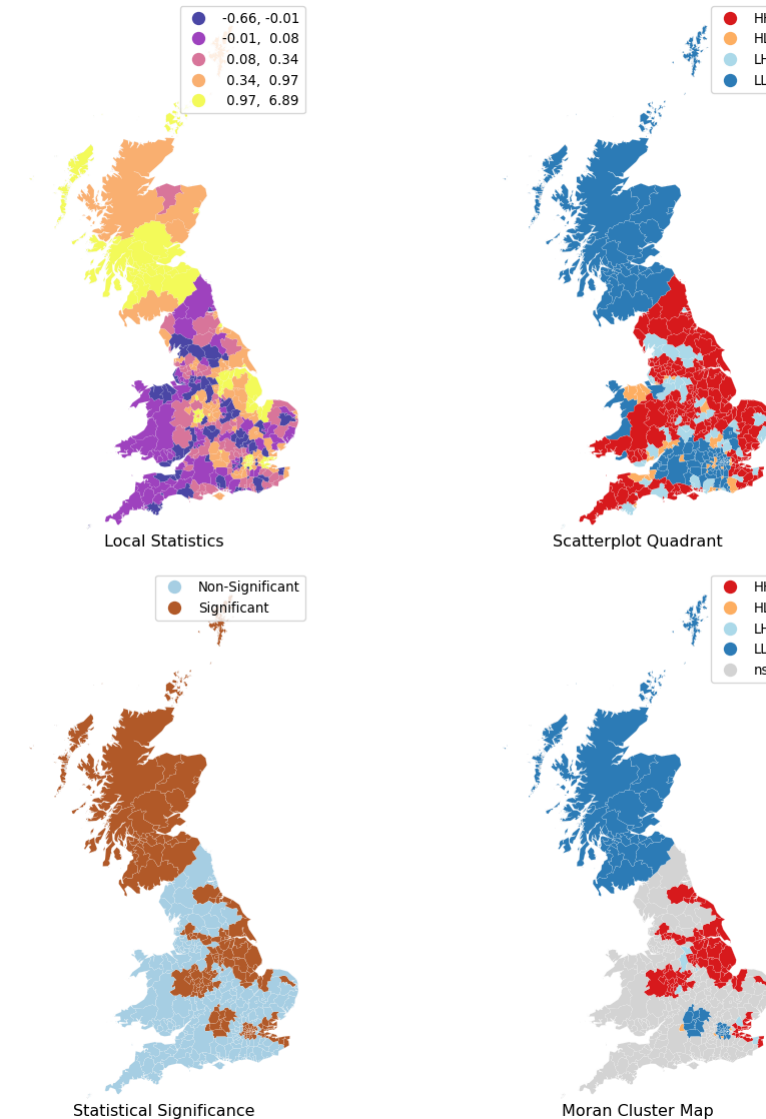
Pour plus de détails: <https://rspatial.org/raster/analysis/3-spauto.html>

# Statistiques spatiales

## Local Indicators of Spatial Association (LISA)

### Algorithme:

1. Définit un seuil de signifiante  $\lambda$
2. Pour chaque nœud:
  1. Calculer l'indice I de Moran pour le groupe formé par le nœud et ses voisins directs
  2. Détermine la signifiante de I
    1. Si  $I > \lambda$  : corrélation positive signifiante
    2. Si  $I < -\lambda$  : corrélation négative signifiante
    3. Si  $-\lambda < I < \lambda$  : pas de corrélation ou corrélation non signifiante

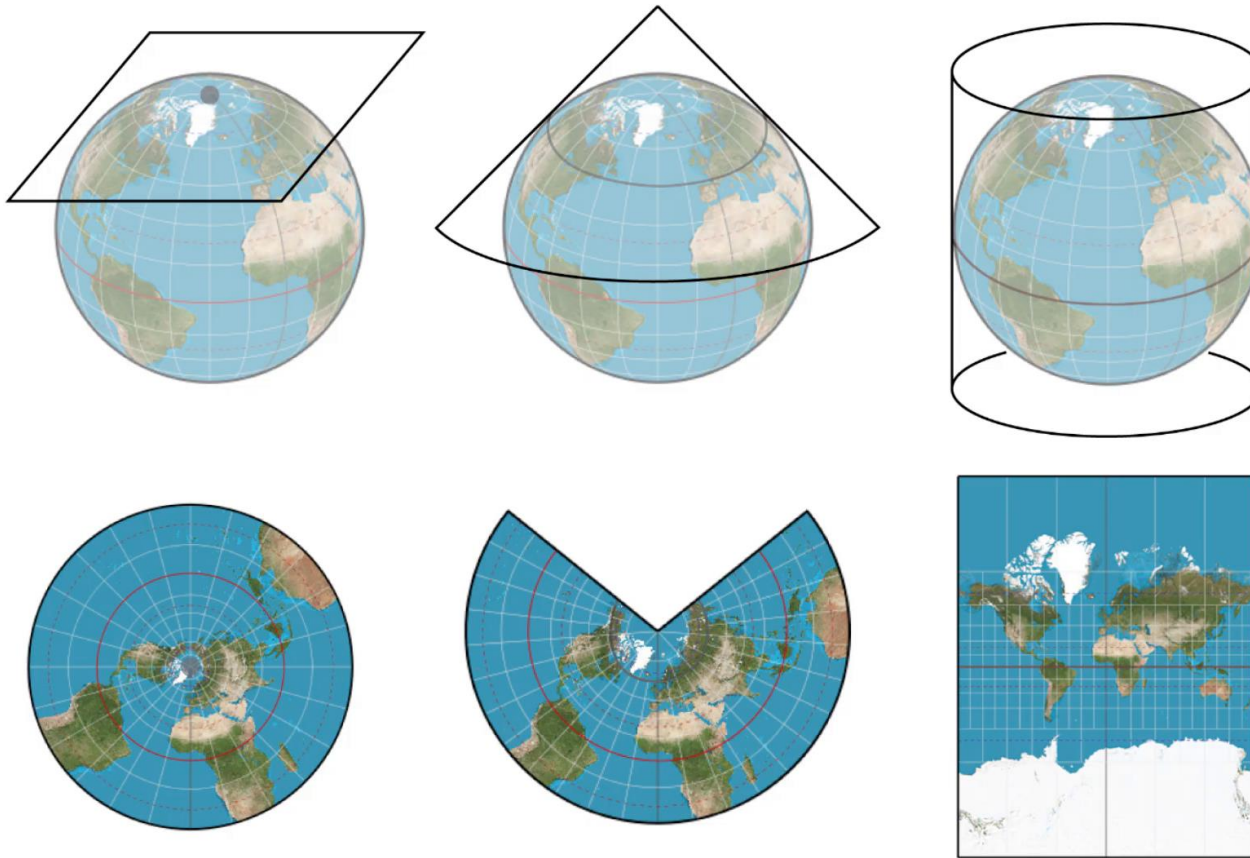


Résultats des votes  
du Brexit

Pour plus de détails: [https://geographicdata.science/book/notebooks/07\\_local\\_autocorrelation.html](https://geographicdata.science/book/notebooks/07_local_autocorrelation.html)

# Petit point avant de finir sur les données spatiales

## L'importance du système de projection

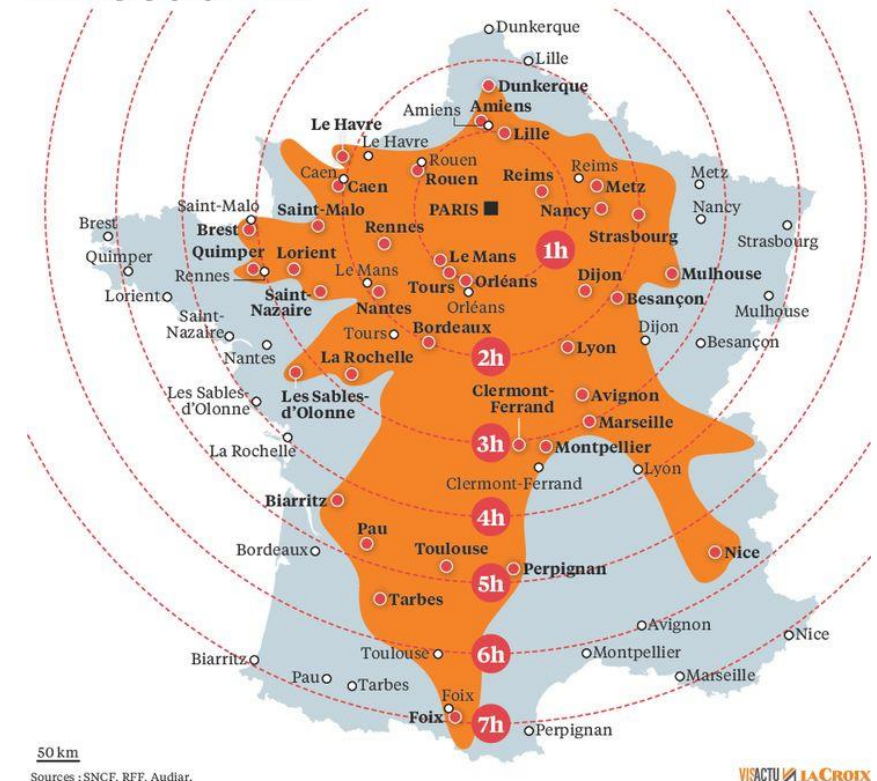


Source: [Introduction aux données spatiales avec Geopandas – Python pour la data science](#)

⇒ Comme pour la régionalisation, il est aussi possible de créer nos propres projections par rapport à une variable d'intérêt

### Les temps de parcours en TGV depuis Paris

o Position géographique des villes



50 km

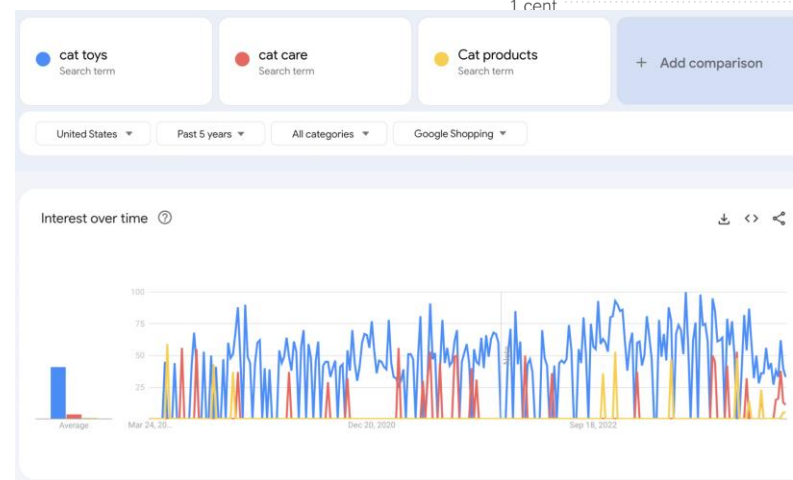
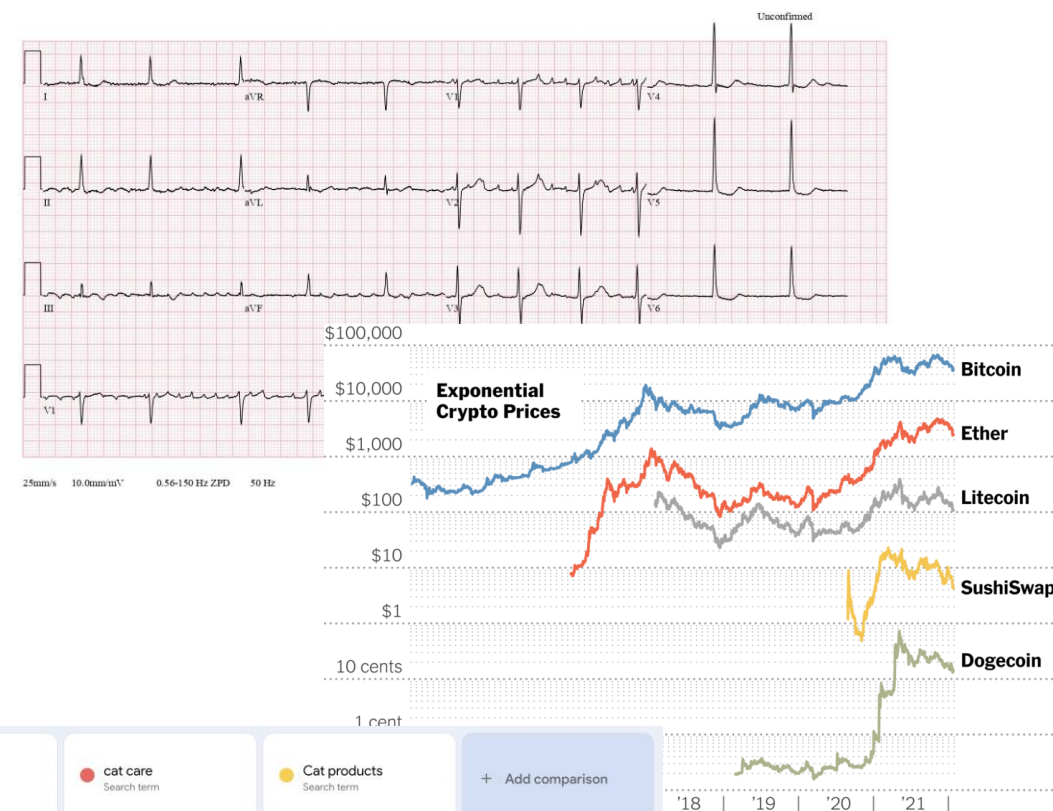
Sources : SNCF, RFF, Audiar.

VISACTU LA CROIX

Lien utile pour comparer: [The True Size of Countries | Compare Real Country Sizes](#)

# Définitions: données temporelles

- Les **données temporelles**, ou ***time series***, ont pour particularité d'être lié à une temporalité:
  - Un « moment »: date, heure, *timestamp*, etc.
  - Une « période »: année, saison, mois, etc.
- Elles décrivent l'évolution d'une ou plusieurs variables sur le temps
- Comme pour les données spatiales, elles nécessitent un traitement particulier



# Miner des données temporelles

Place à la pratique !

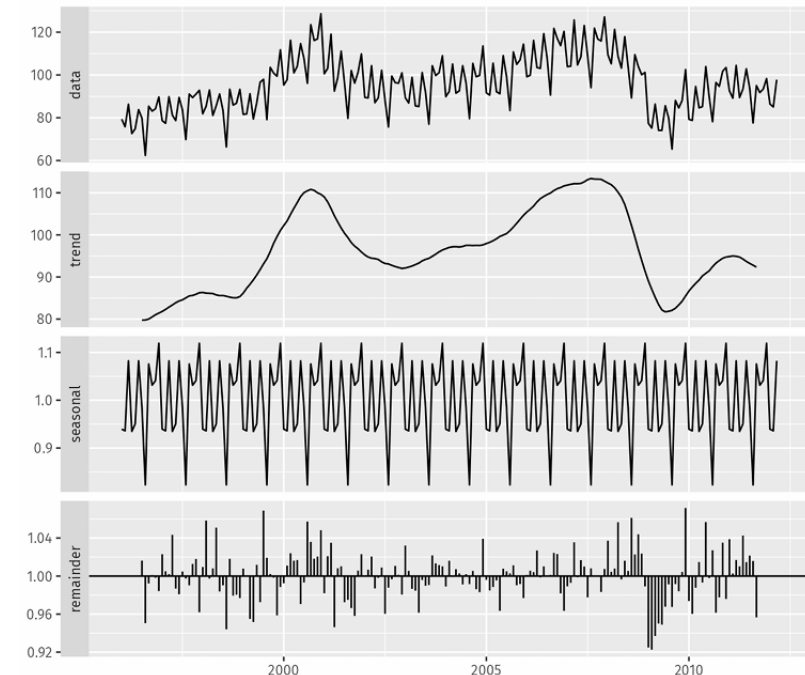
- Sur [kaggle](https://www.kaggle.com/), téléchargez l'un des jeux de données suivants:
  1. [ECG dataset](#)
  2. [Wind & Solar Energy Production Dataset](#)
  3. [House Hold Energy Data - Time Series](#)
  4. [Daily Climate time series data](#)
  5. [StackOverflow Questions Count Time Series](#)
  6. [Temperature Time-Series for some Brazilian cities](#)
  7. [Walmart Sales Dataset of 45stores](#)
- Vérifiez et nettoyez vos données !
- Discutez des informations que vous pouvez extraire de ces données
  - Comment les représenter ?
  - Que peut on repérer dessus et comment ?
- Faites des tests et discutez de vos résultats



# Données temporelles

## Identifier les composantes de l'évolution d'une variable

- Le type d'erreur d'interprétation qu'on ne veut pas commettre:
  - [Les contaminations reculent partout : que disent les indicateurs ? | TF1 Info](#)
- Une variable peut évoluer :
  - En suivant une **tendance (trend)** générale, qui peut augmenter, diminuer ou être constante
  - En suivant une **saisonnalité**, avec des patterns réguliers calqués sur des périodes connues (années, mois, saisons, etc.)
  - En suivant un **cycle**, avec des patterns réguliers non fixés sur des périodes connues
  - De manière **irrégulière, aléatoire**, ou **bruitée**, sans aucun patterns ou tendance apparente
- L'évolution d'une donnée réelle est souvent un assemblage de ces différentes **composantes**
  - ⇒ Il nous faut donc décomposer le signal

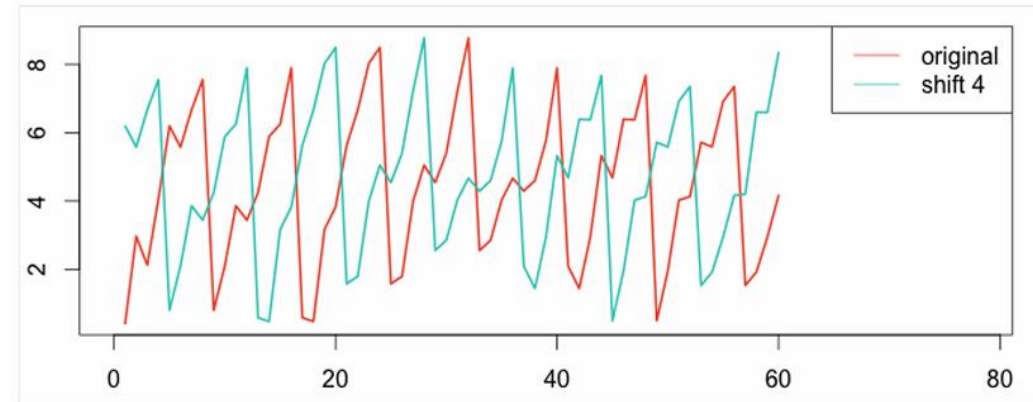


# Données temporelles

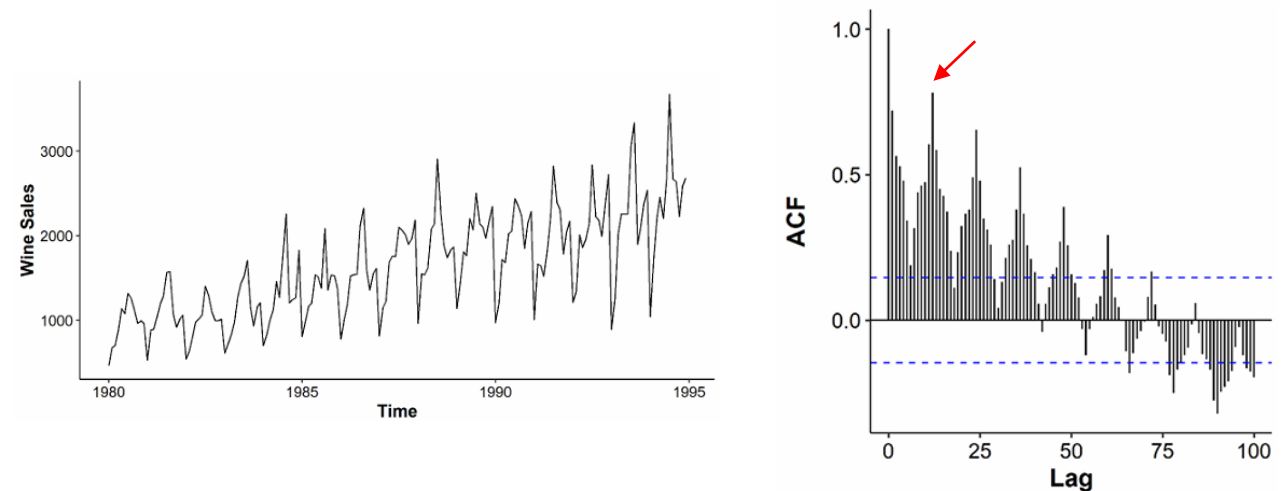
## Auto-correlation et saisonnalité

- **Objectif:** déterminer si une valeur à un instant  $t$  est corrélée avec une valeur à un instant  $t + \Delta_t$ 
  - Contrairement aux données spatiales, on ne regarde le « voisinage » d'une donnée
  - mais on suppose l'existence d'un « **décalage** »  $\Delta_t$ , ou *lag*, sur lequel nous pouvons observer des patterns répétés
  - Permet donc de **repérer une saisonnalité**
- **Approche classique:** calcul de corrélation linéaire (Pearson) entre la *time serie* étudiée et cette même *time serie* décalée de  $\Delta_t$
- **L'Auto Correlation Function (ACF)**, calcule un score d'autocorrelation pour chaque décalage  $\Delta_t$

Approche classique



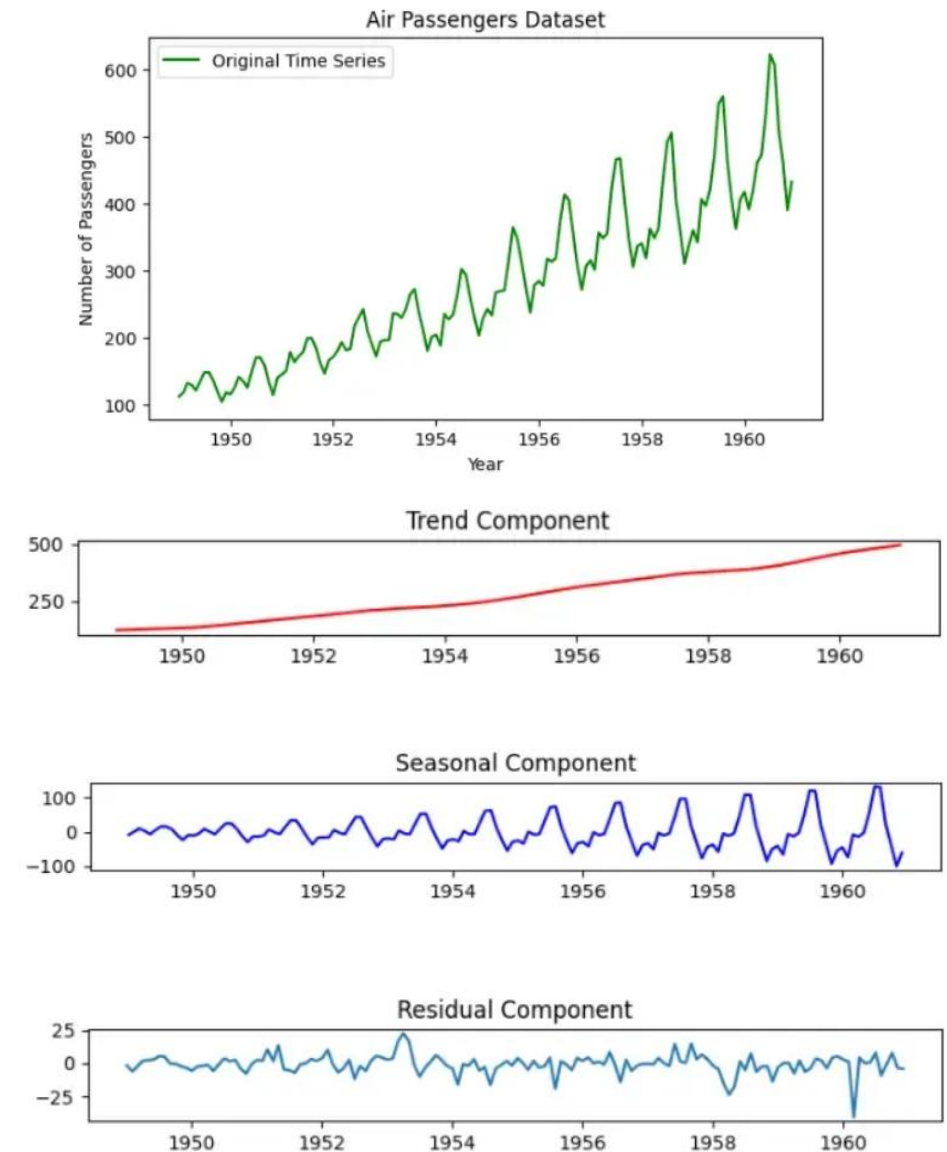
ACF



# Données temporelles

## Décomposition (approche classique)

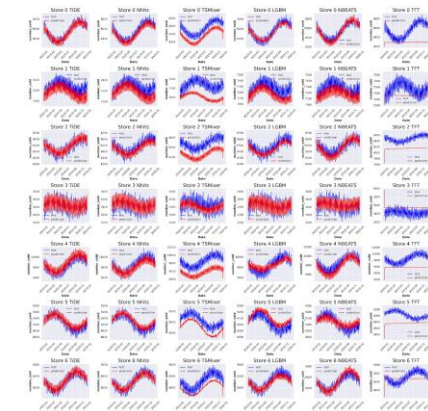
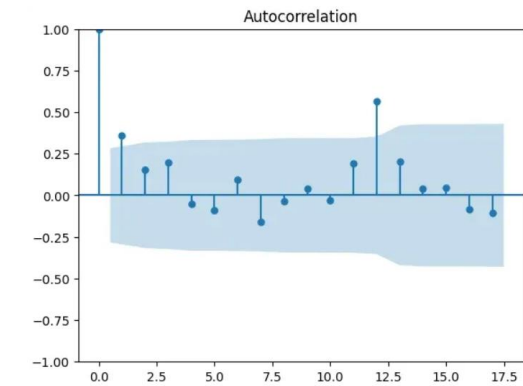
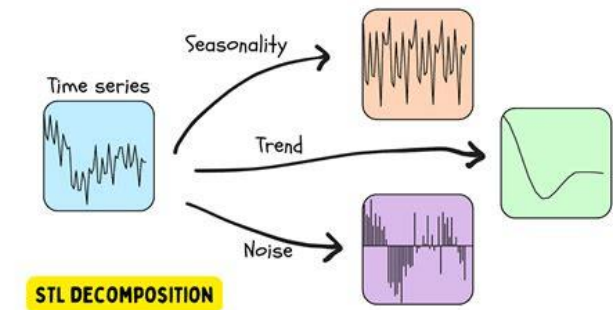
1. Définir une période  $\Delta_t$  pertinente selon ce que vous observez (ex. année, mois, période d'un cycle visible, etc.)
2. Calculer la tendance de la courbe sur une fenêtre glissante de taille  $\Delta_t$  pour obtenir la **composante de tendance** (*trend component*)
  - Ex. régression linéaire
3. Recalculer la courbe en soustrayant la tendance aux valeurs de la variable pour obtenir la **composante saisonnière** (*seasonal component*)
4. Calculer la valeur moyenne pour chaque fenêtre de taille  $\Delta_t$  (aka les « moyennes de saison »)
5. Soustraire les moyennes de saison à la composante saisonnière pour obtenir la **composante résiduelle**



# Données temporelles

## Décomposition (autres algorithmes)

- Seasonal Trend decomposition using LOESS (STL)
  - [https://www.statsmodels.org/devel/examples/notebooks/generated/stl\\_decomposition.html](https://www.statsmodels.org/devel/examples/notebooks/generated/stl_decomposition.html)
- Seasonal Autoregressive Integrated Moving Average (SARIMA)
  - <https://www.geeksforgeeks.org/machine-learning/sarima-seasonal-autoregressive-integrated-moving-average/>
- Facebook Prophet
  - <https://www.datacamp.com/tutorial/facebook-prophet>

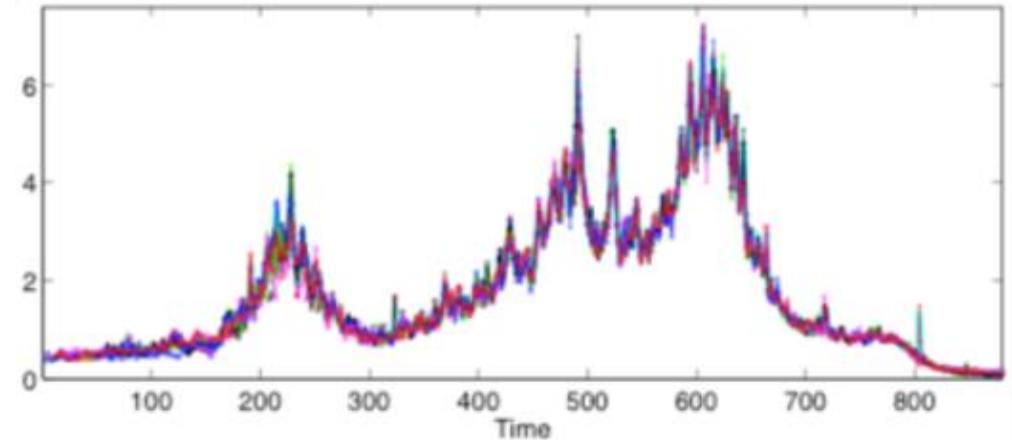


# Données temporelles

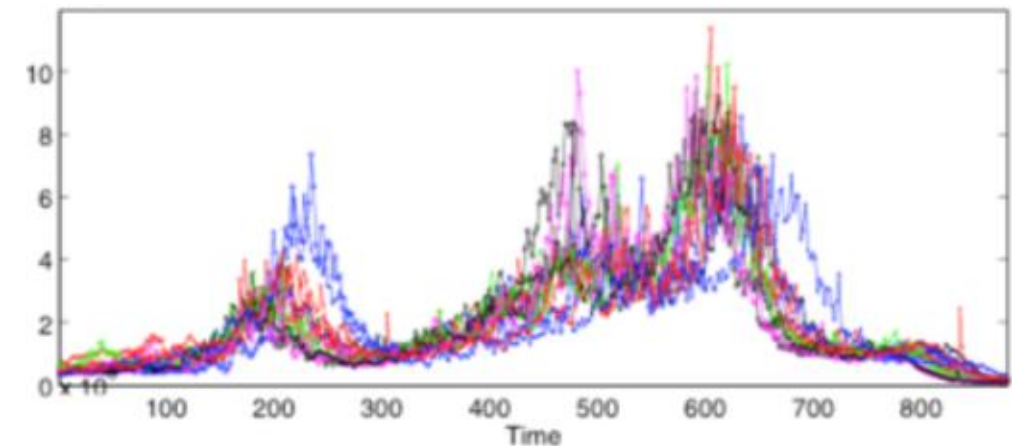
## Regroupement

- **Objectif:** repérer les *time series* similaires
  - Ex. les nombres de ventes par semaine pour différents produits qui suivent une même saisonnalité
- Si les *time series* sont bien « alignées » (même début, même fin, même granularité):
  - Chaque *time serie* peut être considérée comme un vecteur
  - Possibilité d'appliquer des méthodes de regroupement directement (k-means, DB-Scan, etc.)
- **Problème:** les *time series* issues de données réelles sont rarement « bien alignées »

Données « bien alignées »



Données réelles



# Données temporelles

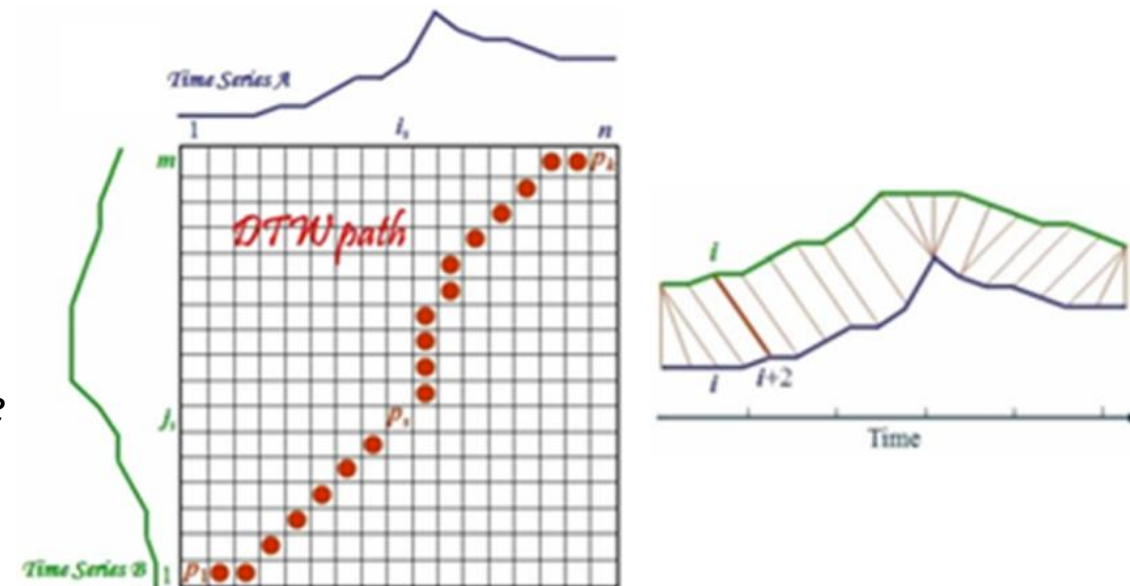
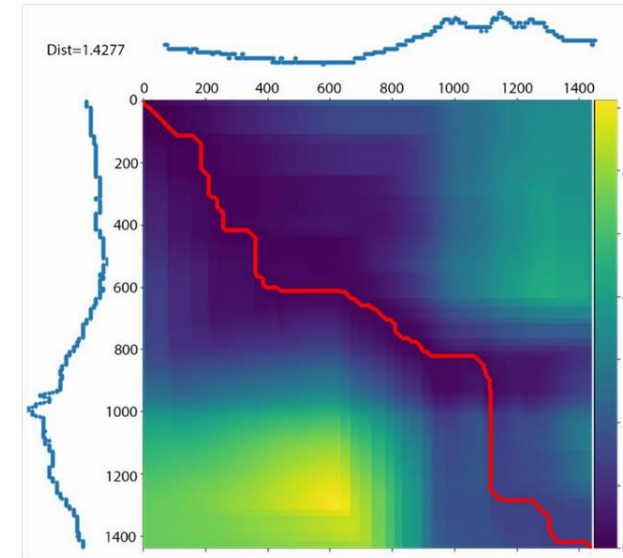
## Réalignement (*Time Warping*)

### Approche « classique »:

1. Construire une matrice de distance pour chaque *timestep* de chaque *time series*
  - Time series de  $m$  et  $n$  mesures
  - matrice de taille  $m \times n$
  - Chaque valeur (distance) de cette matrice sont des « pénalités »
2. Trouver le chemin optimale dans cette matrice (en rouge sur l'exemple), tel qu'il:
  1. Minimise la somme des pénalités
  2. Forme une ligne continue
  3. Et monotone (ne monte jamais)
3. Réaligner les points des *times series* en se basant sur ce chemin

### Problèmes:

- Trouver un chemin plus court -> très coûteux pour de longues *time series*
- Approche exacte: Dijkstra algorithm
- Approche « greedy / lazy »: FastDTW, A\*, etc.



# Données temporelles

## Détection d'anomalies

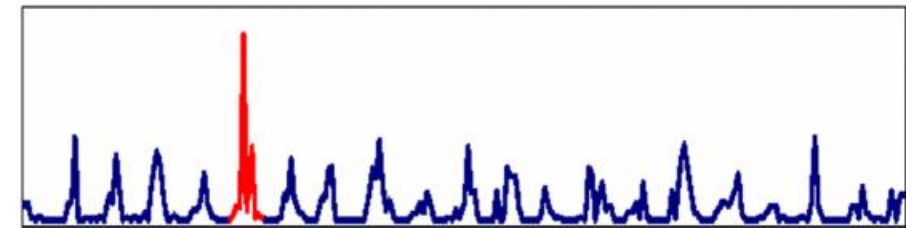
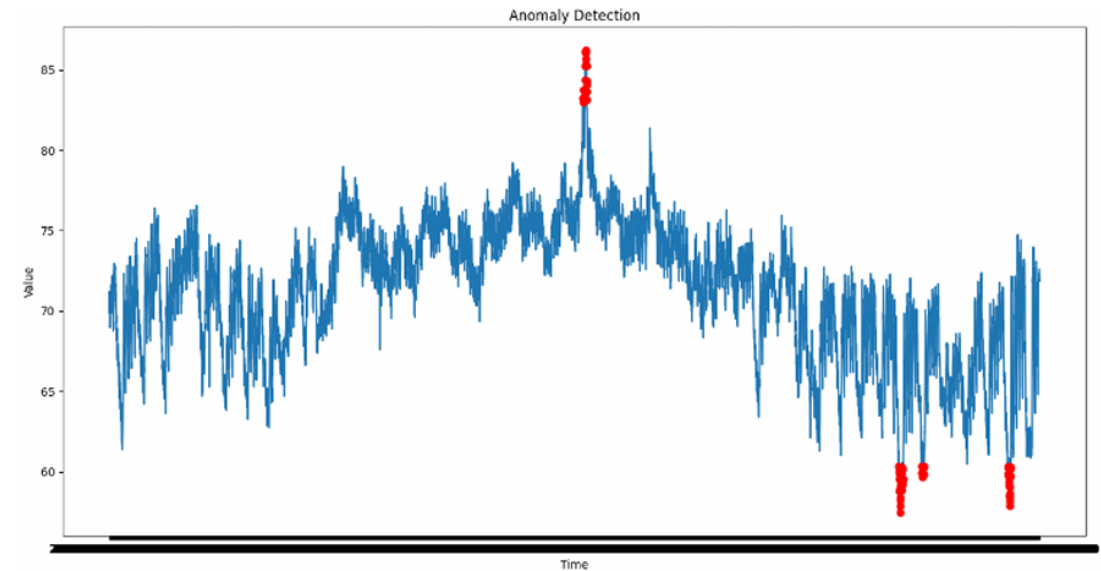
### Approche simple par fenêtre glissante:

1. Choisir une période  $\Delta_t$
2. Pour chaque point  $t$ , nous avons pour référence tous les points dans l'intervalle:  

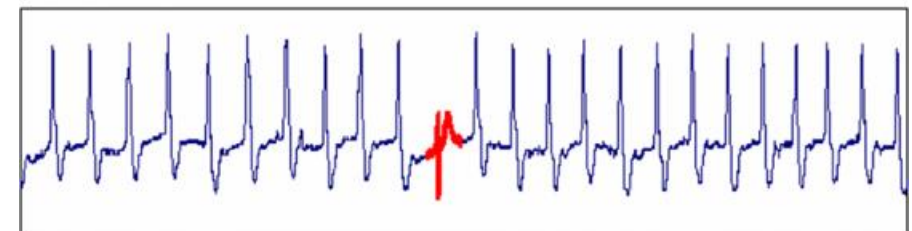
$$\left[t - \frac{\Delta_t}{2}, t + \frac{\Delta_t}{2}\right]$$
3. Déterminer si le point  $t$  est une valeur aberrante (Cf. cours 3)

### Limitée sur des cas complexes (Fig. a et b)

- Peut nécessiter l'utilisation de méthodes plus poussées (Cf. cours 3)



(a)



(b)

# Place à l'action !



## Votre Mission:

- Reprendre votre code là vous l'aviez laissé
  - Lien initial: <https://github.com/a-t-richard/20262027-UE-DataMining-Project>
- Utiliser les nouvelles notions et méthodes vues pendant ce cours
- Trouver ce qu'il se cache dans ces données !